

07 — Apprentissage automatique & théorie de la généralisation

Table des matieres

[5.1 — Features causales & fuite de données \(leakage\)](#)

[5.2 — Normalisation apprise sur le train](#)

[5.3 — Split temporel vs shuffle](#)

[5.4 — Log-loss & règle de score propre](#)

[5.5 — Plancher d'entropie de Bernoulli](#)

[5.6 — Score de Brier](#)

[5.7 — hit@6](#)

[5.8 — Test apparié \(Wilcoxon signed-rank\)](#)

[5.9 — Baselines \(constante · fréquences · logistique\)](#)

[5.10 — Régression logistique](#)

[5.11 — Perceptron multicouche \(MLP\)](#)

[5.12 — Importance par permutation](#)

[5.13 — LSTM & séquences multi-hot](#)

[5.14 — Sur-apprentissage, généralisation & le « mirage »](#)

[Réponses aux exercices](#)

Niveau : base → avancé (tout traité en profondeur). **Prérequis :** 01_fondations.md (Bernoulli, espérance/variance), 03_inference_frequentiste.md (§2.2 p-value, §2.9 permutation), 05_biais_cognitifs.md (illusion de motif). **Couvre les fiches source :** 5.1 à 5.14 de concepts_FR.md. **Source des chiffres :** reports/phase2_ml_FR.md (figures fig08–fig09).

Idée directrice. La Phase 1 (document 06) a montré, par des tests classiques, qu'on ne distingue pas l'historique d'un pur hasard. La Phase 2 le prouve **autrement** : on lâche des modèles de plus en plus puissants (constante → fréquences → logistique → MLP → LSTM) et on vérifie qu'**aucun ne bat le plancher d'entropie hors-échantillon**. L'effondrement *in-sample* → *out-of-sample* du LSTM **est** la démonstration. Mais cette preuve n'a de valeur que si le protocole est irréprochable : pas de fuite du futur, split temporel, métriques honnêtes, tests appariés. D'où l'ordre de ce document : d'abord la **discipline de données**, puis les **métriques**, puis les **modèles**.

Note de lecture. Les fiches sont présentées dans l'ordre numérique de la source (5.1 à 5.14) pour la navigation. Une dépendance mineure : les *baselines* (5.9) s'appuient sur la *régression logistique* (5.10), traitée juste après.

5.1 — Features causales & fuite de données (leakage)

En une phrase

Une feature au temps t ne doit utiliser que l'information **antérieure à t** ; sinon on injecte la réponse dans la question — c'est la **fuite de données** (leakage).

Intuition

Prédire, c'est décider **sans connaître l'avenir**. Si, pour prédire le tirage t , une feature contient ne serait-ce qu'un soupçon d'information sur le tirage t lui-même (ou après), le modèle « triche » : il mémorise la réponse au lieu de l'inférer. Les résultats paraissent excellents... jusqu'au déploiement réel, où l'avenir n'est pas disponible. Le leakage est l'erreur la plus sournoise du ML, car elle gonfle les performances *en apparence*.

Formalisation

Une feature $x_{t,j}$ pour la cellule (tirage t , numéro j) est **causale** si elle est une fonction de l'historique strict $\{\text{tirages } 1, \dots, t - 1\}$ uniquement. Toute dépendance en $\{t, t + 1, \dots\}$ est une **fuite**.

Les 5 features causales du projet (toutes $< t$) :

- gap : nombre de tirages depuis la dernière sortie de j ;
- streak : nombre de présences consécutives de j ;
- rollfreq : fréquence de j sur les 50 derniers tirages ;
- cumfreq : fréquence cumulée de j jusqu'à $t - 1$;
- number_value : $j/49$ (statique, sans temps — donc trivialement causale).

Formes subtiles de leakage (au-delà du cas évident) :

- **Normalisation globale** : standardiser avec une moyenne/écart-type calculés sur *tout* le jeu (train + test) fait fuiter des statistiques du futur (§5.2).
- **Sélection de features** ou réglage d'hyperparamètres sur l'ensemble complet.
- **Cible décalée** : utiliser cumfreq *incluant* t au lieu de $t - 1$.

Tests anti-fuite : vérifier que permuter le futur ne change pas les features passées ; qu'aucune feature n'est anormalement prédictive ; que la performance s'effondre si l'on décale délibérément la cible.

Dans iAlexMG

Le panel est constitué de $(4\,377 - 1) \times 49 = \mathbf{214\,424}$ lignes (le 1^{er} tirage est exclu, faute d'historique). Les 5 features sont **strictement** $< t$, garantissant zéro fuite — condition *sine qua non* pour que le verdict « aucun modèle ne bat le plancher » soit crédible.

Pièges & malentendus

- **Leakage = performance « trop belle »**. Un modèle qui bat un plancher théorique hors-échantillon trahit presque toujours une fuite (ou un bug), pas une découverte.
- **Croire qu'une feature statique fuit**. `number_value` ne dépend pas du temps : pas de fuite possible.
- **Oublier le décalage $t - 1$ vs t** . L'erreur d'un cran transforme une feature causale en fuite.

Pour vérifier ta compréhension

1. Pourquoi `cumfreq` doit-elle s'arrêter à $t - 1$ et non inclure t ?
2. Donne un exemple de leakage qui ne vient *pas* directement d'une feature.
3. Pourquoi un modèle qui « bat le plancher » OOS doit-il éveiller les soupçons ?

(Réponses [à la fin](#).)

Pour aller plus loin

- **Pipelines scikit-learn** : encapsuler la normalisation pour qu'elle soit apprise *dans* la validation croisée (§5.2).
- **Validation temporelle** (§5.3) : la seule défense structurelle contre le leakage en séries.

5.2 — Normalisation apprise sur le train

En une phrase

La moyenne et l'écart-type qui standardisent les features doivent être estimés **sur le train seul**, puis appliqués au test — sinon on fait fuiter le futur.

Intuition

Le jour J , pour standardiser une donnée, on ne dispose que des statistiques du **passé**. Calculer la moyenne sur l'ensemble (train + test) reviendrait à utiliser des informations qui n'existaient pas encore — une fuite subtile mais réelle.

Formalisation

On standardise $\tilde{x} = (x - \hat{\mu}_{\text{train}}) / \hat{\sigma}_{\text{train}}$, où $\hat{\mu}_{\text{train}}, \hat{\sigma}_{\text{train}}$ sont estimés **uniquement** sur le train (§0.4 pour les moments). On applique ces **mêmes** constantes au test, sans les recalculer. En pratique : `StandardScaler.fit(train)` puis `.transform(test)`.

Pourquoi c'est du leakage sinon. La moyenne globale incorpore les valeurs du test (donc du futur) ; même infime, cette contamination invalide l'évaluation hors-échantillon. C'est une forme de fuite **par normalisation** (§5.1).

Dans iAlexMG

Les modèles à features (logistique, MLP) utilisent `StandardScaler` **ajusté sur le train seul** (≤ 2020), appliqué au test (> 2020). Garantit l'honnêteté du log-loss OOS.

Pièges & malentendus

- **fit sur tout le jeu.** Erreur classique ; utiliser `fit(train)` puis `transform`.
- **Re-standardiser le test sur lui-même.** Introduit une incohérence d'échelle entre train et test.

Pour vérifier ta compréhension

1. Pourquoi ne peut-on pas standardiser avec la moyenne de tout le jeu de données ?
2. Quelles statistiques applique-t-on au test ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Pipelines** : intègrent `fit/transform` proprement dans la validation croisée.
- **Normalisation par fenêtre glissante** : variante encore plus stricte pour les séries.

5.3 — Split temporel vs shuffle

En une phrase

On entraîne sur le passé (≤ 2020) et on teste sur le futur (> 2020), **sans jamais mélanger** — un shuffle ferait fuiter le futur via les features cumulatives.

Intuition

Évaluer un prédicteur, c'est l'entraîner sur ce qu'on **savait** et le juger sur ce qui est **venu après**. Mélanger les tirages (shuffle) placerait des tirages futurs dans le train : les features cumulatives (cumfreq, rollfreq) porteraient alors de l'information sur des tirages censés être « à prédire ». Pour des séries temporelles, le shuffle est **interdit**.

Formalisation

Split temporel : un instant de coupure t^* sépare train ($t < t^*$) et test ($t \geq t^*$). Aucune ligne de test n'influence l'entraînement. À l'inverse, un split aléatoire (shuffle) suppose des observations **échangeables** — hypothèse fausse ici, car les features dépendent de l'ordre.

Dans iAlexMG

- **Train ≤ 2020 : 188 846 lignes ; test > 2020 : 25 578 lignes.**
- **Jamais de shuffle** : les features cumulatives rendraient un mélange équivalent à du leakage (§5.1).
- C'est le **protocole d'évaluation de toute la Phase 2**.

Pièges & malentendus

- **Validation croisée k-fold standard sur des séries**. Le shuffle implicite fuit ; utiliser une CV **temporelle** (rolling/expanding).
- **Croire le split temporel « gaspilleur »**. Il est la seule évaluation honnête pour la prédiction.

Pour vérifier ta compréhension

1. Pourquoi un shuffle est-il dangereux avec cumfreq ?
2. Combien de lignes au train et au test, et selon quel critère sont-elles séparées ?

(Réponses [à la fin](#).)

Pour aller plus loin

- **Validation croisée temporelle** (rolling/expanding windows) : robustifie l'estimation OOS.
- **Backtesting** : la version « finance » du split temporel.

5.4 — Log-loss & règle de score propre

En une phrase

Le log-loss pénalise les probabilités prédites ; c'est une **règle de score propre** : son espérance est minimale **exactement** quand la probabilité prédite égale la vraie probabilité — impossible de tricher en bluffant.

Intuition

On veut une métrique qui récompense l'honnêteté probabiliste. Le log-loss est ce juge incorruptible : annoncer une proba qu'on ne croit pas augmente *en espérance* sa pénalité. Le seul moyen d'optimiser son score est de dire la vérité sur la probabilité.

Formalisation

Pour une cible binaire $y \in \{0,1\}$ et une proba prédite q , le log-loss (base e , en nats) est

$$\ell(y, q) = -[y \ln q + (1 - y) \ln(1 - q)],$$

moyenné sur toutes les observations. C'est l'**entropie croisée** entre la cible et la prédiction, et l'opposé de la log-vraisemblance de Bernoulli.

Preuve qu'il est « propre ». Si la vraie probabilité est p et qu'on prédit q , l'espérance du log-loss vaut

$$\mathbb{E}_p[\ell] = -[p \ln q + (1 - p) \ln(1 - q)].$$

Dérivons par rapport à q et annulons :

$$\frac{d}{dq} \mathbb{E}_p[\ell] = -\left(\frac{p}{q} - \frac{1-p}{1-q}\right) = 0 \Leftrightarrow p(1-q) = q(1-p) \Leftrightarrow q = p.$$

La dérivée seconde est positive : c'est bien un **minimum**, atteint à $q = p$. Donc aucune prédiction $q \neq p$ ne peut faire mieux en espérance — la définition d'une **règle de score propre**. La valeur minimale est l'entropie $H(p)$ (§5.5).

Clip numérique. Comme $\ln 0 = -\infty$, on borne $q \in [\varepsilon, 1 - \varepsilon]$ (petit ε) pour éviter les pertes infinies sur une prédiction extrême erronée.

Dans iAlexMG

- **Métrique-reine de la Phase 2** : tous les modèles sont comparés en log-loss OOS au **plancher** $H(6/49) \approx 0,3718$ nat (§5.5).
- Base e (nats) pour la cohérence avec l'entropie.

Pièges & malentendus

- **Optimiser le log-loss in-sample.** On peut le faire baisser en mémorisant (§5.14) ; seul l'OOS compte.

- **Oublier le clip.** Une seule prédiction $q = 0$ sur un $y = 1$ envoie le log-loss à l'infini.
- **Comparer des bases différentes.** Nats (base e) vs bits (base 2) diffèrent d'un facteur $\ln 2$; rester cohérent.

Pour vérifier ta compréhension

1. Montre que $\mathbb{E}_p[\ell]$ est minimisé en $q = p$.
2. Pourquoi a-t-on besoin d'un clip $[\varepsilon, 1 - \varepsilon]$?
3. Quel lien entre log-loss et log-vraisemblance de Bernoulli ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Divergence de Kullback-Leibler** : $\mathbb{E}_p[\ell] = H(p) + \text{KL}(p \parallel q)$; le surcoût par rapport au plancher est exactement la KL.
- **Autres règles propres** : Brier (§5.6), score logarithmique, score sphérique.

5.5 — Plancher d'entropie de Bernoulli

En une phrase

Le meilleur log-loss atteignable sous H_0 est l'entropie binaire de $p = 6/49$, soit $H(p) \approx 0,3718$ nat — la barre que **personne** ne peut franchir si le tirage est vraiment aléatoire.

Intuition

Si les tirages sont un pur hasard à probabilité $6/49$, la meilleure prédiction possible est $6/49$ pour tout le monde, et son log-loss est l'entropie de cette pièce truquée. Aucun modèle honnête ne peut faire mieux **durablement** : « battre le plancher » hors-échantillon signifierait avoir trouvé une structure inexistante (donc tricher ou sur-apprendre).

Formalisation

L'**entropie binaire de Shannon** de p (la valeur minimale du log-loss attendu, §5.4) :

$$H(p) = -p \ln p - (1 - p) \ln(1 - p).$$

Pour $p = 6/49$:

$$H(6/49) = -6/49 \ln 6/49 - 43/49 \ln 43/49 \approx 0,2571 + 0,1146 = \mathbf{0,3718} \text{ nat.}$$

Pourquoi c’est un plancher. Par la décomposition $\mathbb{E}_p[\ell] = H(p) + \text{KL}(p \parallel q) \geq H(p)$ (la divergence KL est ≥ 0 , nulle ssi $q = p$). Donc le log-loss attendu est **borné inférieurement par $H(p)$** , atteint uniquement par la vraie probabilité.

Nats vs bits. En base 2, $H_2(6/49) = H(6/49)/\ln 2 \approx 0,536$ bit. Le projet travaille en **nats** (base e) pour coller au log-loss naturel.

Dans iAlexMG

- **Référence absolue** de toutes les figures 8–11 : chaque modèle est jugé à l’aune de 0,3718.
- La **baseline constante** ($p = 6/49$ partout) **atteint le plancher pile** (0,3718) — elle prédit le vrai taux de base, donc rien ne peut la battre sous H_0 .

Pièges & malentendus

- **Croire qu’on peut le battre avec « plus de calcul ».** Sous H_0 , le plancher est indépassable ; plus de capacité n’achète aucun pouvoir prédictif (§5.14).
- **Confondre in-sample et OOS.** On *peut* passer sous le plancher in-sample (en mémorisant), jamais OOS de façon stable.
- **Mélanger nats et bits.** Toujours préciser la base.

Pour vérifier ta compréhension

1. Recalcule $H(6/49)$ et convertis en bits.
2. Pourquoi la baseline constante atteint-elle exactement le plancher ?
3. Que signifierait un modèle sous le plancher en OOS ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Entropie de Shannon** : mesure générale d’incertitude ; le plancher en est le cas binaire.
- **Lien KL** (§5.4) : le surcoût d’un modèle imparfait au-dessus du plancher = sa divergence à la vérité.

5.6 — Score de Brier

En une phrase

Le score de Brier est la moyenne des carrés d'écart $(q - y)^2$: une **seconde règle de score propre**, quadratique et bornée, qui sert de contre-vérification au log-loss.

Intuition

Là où le log-loss punit sévèrement (logarithmiquement) les prédictions confiantes et fausses, le Brier punit plus doucement (quadratiquement) et reste borné dans $[0,1]$. Deux juges propres qui concordent renforcent la conclusion.

Formalisation

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (q_i - y_i)^2.$$

C'est une **règle propre** (minimisée en espérance à $q = p$, même raisonnement qu'en §5.4).

Décomposition utile (Murphy) en **calibration** (les probas annoncées correspondent-elles aux fréquences réalisées ?) et **raffinement** (résolution). Sa borne $[0,1]$ le rend lisible.

Dans iAlexMG

Colonne de **contrôle** des tableaux OOS : toutes les baselines affichent un Brier $\approx 0,1075$, indiscernables — confirmant le verdict du log-loss par une métrique indépendante.

Pièges & malentendus

- **Croire Brier plus « doux » donc moins fiable.** Il est tout aussi propre ; sa pénalisation quadratique le rend juste moins sensible aux prédictions extrêmes.
- **Comparer Brier et log-loss en valeur absolue.** Échelles différentes ; comparer chacun à sa propre référence.

Pour vérifier ta compréhension

1. Pourquoi le Brier est-il borné alors que le log-loss ne l'est pas ?
2. Que vérifie-t-on quand log-loss et Brier concordent ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Décomposition calibration-raffinement** : diagnostiquer *pourquoi* un modèle score mal.
 - **Diagrammes de fiabilité** : visualiser la calibration.
-

5.7 — hit@6

En une phrase

hit@6 compte, par tirage, combien des 6 numéros les mieux notés sont réellement tirés ; sa référence hasard est $6 \times 6/49 \approx 0,735$.

Intuition

Une métrique « métier », lisible pour une vidéo : « sur les 6 numéros que le modèle privilégie, combien sortent ? ». Intuitive, mais **bruitée** sur peu de tirages — d'où son rôle illustratif, jamais d'arbitre.

Formalisation

Pour chaque tirage, on classe les 49 numéros par proba prédite, on prend le top-6, et on compte combien figurent parmi les 6 tirés. Sous H_0 (toutes probas égales), l'espérance est $6 \times \frac{6}{49} \approx 0,735$. Sur m tirages test, l'écart-type de la moyenne décroît en $1/\sqrt{m}$ (§0.4) — vite grand quand m est petit.

Dans iAlexMG

- Référence hasard $\approx 0,735$; les baselines tournent autour (constante 0,680, logistique 0,730).
- **Bruité** : le LSTM finit à hit@6 = 0,747 OOS (au-dessus du hasard) **mais oscille de 0,67 à 0,77** sur ~522 tirages test — du **bruit d'échantillon**, pas un avantage. Le log-loss et le test apparié (robustes) tranchent dans l'autre sens.

Pièges & malentendus

- **Lire un hit@6 > 0,735 comme une victoire.** Sur peu de tirages, le bruit domine ; l'arbitre reste le log-loss.
- **Oublier l'incertitude.** Toujours regarder l'oscillation, pas la valeur finale.

Pour vérifier ta compréhension

1. D'où vient la référence 0,735 ?
2. Pourquoi hit@6 = 0,747 du LSTM ne contredit-il pas « le LSTM est pire » ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Métriques de classement** (precision@k, MAP) : famille à laquelle hit@6 appartient.
- **Intervalles de confiance** sur hit@6 par bootstrap pour quantifier le bruit.

5.8 — Test apparié (Wilcoxon signed-rank)

En une phrase

Le test apparié compare deux modèles sur les **mêmes tirages** (pertes appariées) ; en neutralisant la difficulté commune des tirages, il isole la vraie différence et gagne en puissance.

Intuition

Comparer deux moyennes de log-loss brutes, c'est laisser le bruit « tirage facile / tirage difficile » masquer la différence entre modèles. En appariant — on regarde, tirage par tirage, *quel* modèle fait mieux — la difficulté commune s'annule, et le signal résiduel est la vraie différence. C'est bien plus puissant.

Formalisation

H_0 : les deux modèles ont la même performance — médiane des différences de pertes médiane(d_i) = 0. H_1 : un modèle fait significativement mieux que l'autre (médiane(d_i) $\neq 0$) — l'écart de log-loss n'est pas du bruit.

Pour chaque tirage i , on calcule la différence de pertes $d_i = \ell_i^A - \ell_i^B$. Le **Wilcoxon signed-rank** teste H_0 : médiane(d_i) = 0 de façon **non paramétrique** (sur les rangs des $|d_i|$ signés), sans supposer la normalité. L'appariement **réduit la variance** : $\text{Var}(d) = \text{Var}(\ell^A) + \text{Var}(\ell^B) - 2 \text{Cov}(\ell^A, \ell^B)$, et la covariance est forte (mêmes tirages), donc $\text{Var}(d)$ est petite.

Dans iAlexMG

« Cet écart de log-loss est-il réel ou du bruit ? » à chaque modèle :

- **Logistique vs constante** : $p = 0,14 \rightarrow$ aucun écart significatif.
- **MLP vs constante** : $p = 0,029 \rightarrow$ significatif, mais le MLP est *pire* (le « piège du significatif », §5.14).
- **LSTM vs constante** : $p < 0,0001 \rightarrow$ le LSTM est significativement **pire**.

Pièges & malentendus

- « **Significatif** » $\Rightarrow$ « **meilleur** ». Le signe compte : ici les écarts significatifs sont *défavorables* aux modèles complexes.
- **Comparer des moyennes non appariées**. Beaucoup moins puissant ; gaspille la structure commune des tirages.

Pour vérifier ta compréhension

1. Pourquoi l'appariement réduit-il la variance de la différence ?
2. MLP vs constante : $p = 0,029$. Le MLP est-il meilleur ? Que conclure ?
3. Pourquoi Wilcoxon plutôt qu'un t apparié ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **t apparié** : alternative paramétrique (suppose la normalité des différences).
 - **Bootstrap apparié** : autre façon d'estimer l'incertitude de l'écart.
-

5.9 — Baselines (constante · fréquences · logistique)

En une phrase

Pour juger un modèle, il faut une barre basse honnête : du plus bête (constante $p = 6/49$) au moins bête (logistique sur features) — et toutes collent au plancher.

Intuition

Sans point de comparaison, un log-loss de 0,372 ne veut rien dire. Les baselines fournissent l'échelle : si un modèle sophistiqué n'égale même pas la **constante**, c'est qu'il n'apporte rien (ou nuit).

Formalisation

Trois baselines de complexité croissante :

- **Constante** : $p = 6/49$ pour toutes les cellules — incarne le plancher.
- **Fréquences** : p_j = fréquence du numéro j apprise sur le train.
- **Logistique** : $P(y = 1)$ via la fonction logistique sur les 5 features (§5.10).

Dans iAlexMG (résultats OOS)

Modèle	log-loss OOS	Brier	hit@6
constante 6/49	0,3718	0,1075	0,680
fréquences	0,3720	0,1075	0,684

Modèle	log-loss OOS	Brier	hit@6
logistique	0,3719	0,1075	0,730
plancher	0,3718	—	0,735

Lecture. Les trois sont **collées au plancher**. La constante l’atteint pile. La logistique est même très légèrement *pire* que la constante (test apparié $p = 0,14 \rightarrow$ non significatif). Sous H_0 , **la constante est imbattable** : prédire le vrai taux de base est optimal.

Pièges & malentendus

- **Sauter les baselines.** Sans elles, impossible d’interpréter un score.
- **Croire « plus complexe = mieux ».** La logistique $\leq$ constante ici.

Pour vérifier ta compréhension

1. Pourquoi la constante est-elle imbattable sous H_0 ?
2. Que dit le fait que la logistique soit $\approx$ à la constante ($p = 0,14$) ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Baseline « taux de base »** : référence universelle en classification déséquilibrée.
- **Modèles nuls** : généralisation de l’idée de barre honnête.

5.10 — Régression logistique

En une phrase

La régression logistique modélise $P(y = 1)$ par une fonction logistique d’une combinaison linéaire des features ; ses coefficients sont interprétables — et celui de $\text{gap} \approx 0$ réfute le sophisme du joueur.

Intuition

Une frontière linéaire « molle » entre sortir et ne pas sortir : on combine linéairement les features, on passe le résultat dans une sigmoïde qui le comprime en $[0,1]$. Le coefficient de chaque feature dit dans quel sens et avec quelle force elle pousse la probabilité.

Formalisation

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Les coefficients β sont estimés par **maximum de vraisemblance**, ce qui revient à **minimiser le log-loss** (§5.4) — d'où le lien direct avec l'entropie croisée. Un coefficient β_k s'interprète en **odds-ratio** : e^{β_k} est le facteur multiplicatif des cotes par unité de x_k . La régularisation (L2/L1) prévient le sur-ajustement.

Dans iAlexMG

- **Baseline 3** (§5.9), log-loss OOS 0,3719.
- **Coefficient gap** ≈ 0 (+0,0017) : le modèle, **libre** d'exploiter le retard d'un numéro, choisit de l'ignorer → ni sophisme du joueur, ni main chaude (§7.1, §7.2).
- Plus gros coefficient : number_value (+0,025), **fantôme** du léger penchant vers les gros numéros (T5, $z \approx 3,2$) — mais inutile (hit@6 < hasard).

Pièges & malentendus

- **Interpréter un gros coefficient comme un signal utile.** number_value est le plus grand et ne sert à rien en prédiction.
- **Oublier la normalisation.** Les coefficients ne sont comparables que sur features standardisées (§5.2).

Pour vérifier ta compréhension

1. Que vaut $\sigma(0)$, et pourquoi est-ce cohérent avec une proba de base ?
2. Pourquoi le coefficient gap ≈ 0 est-il une réfutation du sophisme du joueur ?
3. Quel lien entre maximiser la vraisemblance et minimiser le log-loss ?

(Réponses [à la fin](#).)

Pour aller plus loin

- **Odds-ratios** : lecture multiplicative des coefficients.
- **Régularisation L1/L2** : sélection de features et stabilité.
- **GLM** : la logistique comme modèle linéaire généralisé (lien logit).

5.11 — Perceptron multicouche (MLP)

En une phrase

Le MLP empile des couches denses et des non-linéarités ; c'est un **approximateur universel** — assez puissant pour épouser n'importe quelle fonction... y compris le bruit.

Intuition

Avec assez de neurones, un MLP peut représenter quasiment n'importe quelle relation. C'est une force (flexibilité) et un danger (il peut mémoriser le bruit). S'il existait un signal non-linéaire dans les features, ce réseau devrait le capter. Ici, il ne capte rien — et c'est le bon résultat.

Formalisation

Un MLP applique des couches $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$ avec une non-linéarité ϕ (ReLU...), la dernière couche produisant un logit passé en sigmoïde (perte BCEWithLogits, équivalente au log-loss, §5.4).

Entraînement par **rétropropagation** (gradient de la perte) et optimiseur **Adam**. Le **théorème d'approximation universelle** garantit qu'un MLP à une couche cachée assez large approxime toute fonction continue — d'où sa **capacité** élevée, à brider pour ne pas sur-apprendre.

Dans iAlexMG

- **Étape 2.2** : MLP PyTorch (2 couches de 64 neurones, BCEWithLogits).
- **Témoin négatif** : log-loss in-sample 0,3719, OOS 0,3721, plancher 0,3718 — **tout collé au plancher** (figure 8b). Avec seulement **5 features faibles**, le MLP n'a **rien à mémoriser** ; le vrai mirage viendra du LSTM (§5.13–5.14), nourri de la séquence complète.

Pièges & malentendus

- « **Plus de neurones = meilleures prédictions** ». Sous H_0 , non : la capacité sert seulement à mémoriser le bruit (§5.14).
- **Confondre capacité et signal**. Le MLP *pourrait* tout apprendre ; s'il n'apprend rien, c'est qu'il n'y a rien.

Pour vérifier ta compréhension

1. Pourquoi le MLP est-il un « témoin négatif » plutôt qu'un échec ?
2. Que dit le théorème d'approximation universelle, et pourquoi est-ce à double tranchant ?
3. Pourquoi le MLP n'a-t-il « rien à mémoriser » ici ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Rétropropagation & Adam** : mécanique de l'optimisation.
 - **Régularisation** (dropout, weight decay, early stopping) : brider la capacité.
 - **Biais-variance** (§5.14) : le cadre théorique du sur-apprentissage.
-

5.12 — Importance par permutation

En une phrase

On mélange une feature dans le test et on mesure la **dégradation** du log-loss OOS ; une dégradation ≈ 0 signe une feature inutile.

Intuition

Si casser une feature (en brouillant ses valeurs) ne change rien à la prédiction, c'est qu'elle ne portait aucune information. C'est une mesure **model-agnostic** : elle juge l'usage réel d'une feature en prédiction, pas la taille de ses poids.

Formalisation

Pour chaque feature k , on **permuté** aléatoirement sa colonne dans le **test** (§2.9), on recalcule le log-loss OOS, et l'**importance** est la dégradation $\Delta_k = \ell_{\text{permuté}} - \ell_{\text{original}}$. Un $\Delta_k \approx 0$ = feature sans valeur prédictive. Avantage sur la lecture des poids : robuste aux corrélations entre features et reflète la performance OOS réelle.

Dans iAlexMG

- **Figure 8a** : en mélangeant chaque feature dans le test, la dégradation **maximale** est **+0,001 nat (gap)** — du **bruit**. **Aucune feature ne porte de signal exploitable.**

Pièges & malentendus

- **Confondre importance par permutation et taille des poids.** Un gros poids (number_value) peut avoir une importance OOS nulle.
- **Permuter le train.** Il faut permuter le **test** pour mesurer l'usage OOS.
- **Ignorer les corrélations.** Des features corrélées peuvent se « couvrir » mutuellement.

Pour vérifier ta compréhension

1. Que signifie une importance par permutation ≈ 0 ?

2. Pourquoi est-elle préférable à la lecture des coefficients ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **SHAP / importances locales** : attribution par prédiction.
 - **Lien §2.9** : la permutation comme outil général de loi nulle.
-

5.13 — LSTM & séquences multi-hot

En une phrase

Le LSTM est un réseau récurrent à mémoire ; nourri de la fenêtre des L derniers tirages en **multi-hot** (49 dimensions par pas), il a enfin assez de capacité pour être tenté de mémoriser.

Intuition

On change de représentation : au lieu de 5 features faibles, on donne au modèle la **séquence brute** des derniers tirages, chacun encodé en vecteur 49-dim (un 1 par numéro tiré). Avec des milliers de paramètres et une mémoire récurrente, le LSTM peut capter — ou fabriquer — des dépendances temporelles. C'est le candidat idéal pour révéler un mirage.

Formalisation

- **Multi-hot** : chaque tirage $\rightarrow$ vecteur $\in \{0,1\}^{49}$ (un 1 par numéro tiré, contrainte « 6 uns », §6.1).
Entrée = fenêtre des $L = 10$ derniers tirages (49-dim/pas) ; sortie = 49 sigmoïdes (proba de chaque numéro au tirage suivant), perte **BCE par numéro** (§5.4).
- **Cellule LSTM** : des **portes** (entrée, oubli, sortie) régulent un état mémoire c_t , permettant de retenir ou d'oublier l'information sur de longues séquences — d'où la grande capacité.

Dans iAlexMG

- **Étape 2.3, la punchline** : LSTM PyTorch (nn.LSTM, 64 unités), $L = 10$.
- Suffisamment de capacité pour **mémoriser les coïncidences du passé** $\rightarrow$ met en scène l'opposition mémorisation / généralisation (§5.14).

Pièges & malentendus

- **Croire que la mémoire récurrente « voit » un vrai signal.** Elle peut tout aussi bien mémoriser du bruit (§5.14).

- **Oublier la contrainte multi-hot.** Exactement 6 uns par tirage (sans remise, §0.3).

Pour vérifier ta compréhension

1. Qu'est-ce qu'un encodage multi-hot d'un tirage, et combien de 1 contient-il ?
2. Pourquoi le LSTM est-il « tenté de mémoriser » là où le MLP ne l'était pas ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Portes LSTM** : mécanique détaillée de l'état mémoire.
- **GRU, Transformers** : autres architectures séquentielles.

5.14 — Sur-apprentissage, généralisation & le « mirage »

En une phrase

Un modèle à forte capacité fait baisser la perte **in-sample** (sous le plancher) sans baisser l'**out-of-sample** (qui stagne ou remonte) : il mémorise sans généraliser — et l'écart train↔test est la preuve.

Intuition

Apprendre par cœur n'est pas comprendre. Un élève qui mémorise les corrigés brille à l'examen connu et échoue au nouveau. Le LSTM fait pareil : il « bat » le plancher sur le passé en mémorisant des coïncidences, puis s'effondre sur le futur. Plus on le laisse apprendre, plus l'illusion grandit — sans jamais gagner un pouce sur l'avenir.

Formalisation

H₀ : le LSTM ne généralise pas mieux que le hasard — sa log-loss **OOS** ne bat pas le plancher $H(6/49) = 0,3718$ (la baisse in-sample ne se transfère pas). **H₁** : le LSTM a capté un vrai signal — sa log-loss **OOS** passe durablement sous le plancher.

Le mirage, mesuré (log-loss train et OOS par epoch, figure 9a) :

epoch	train	test
1	0,3721	0,3727
11	0,3714	0,3722

epoch	train	test
31	0,3689	0,3732
50	0,3659	0,3754

La courbe d'entraînement **plonge sous le plancher** ($0,3659 \ll 0,3718$) — mémorisation des coïncidences. La courbe hors-échantillon **remonte au-dessus** ($0,3754$) et continue de grimper — **effondrement**. L'écart $\text{train} \leftrightarrow \text{test}$ se creuse epoch après epoch : la signature du sur-apprentissage **sur du bruit**. Test apparié : LSTM significativement pire que la constante, $p < 0,0001$.

Cadre théorique (biais-variance). L'erreur de généralisation se décompose en biais², variance et bruit irréductible. Augmenter la capacité réduit le biais mais **gonfle la variance** ; sous H_0 (aucun signal), il n'y a *rien à apprendre*, donc toute baisse in-sample est de la variance pure qui ne se transfère pas au test. Les **courbes d'apprentissage** (train vs OOS) rendent ce phénomène visible. **Conclusion fondamentale** : sous H_0 , plus de calcul n'achète aucun pouvoir prédictif.

Dans iAlexMG

- **Cœur narratif de la Phase 2** (figure 9) : opposition in/out systématique.
- **Punchline** : *le modèle le plus sophistiqué échoue le plus franchement. Si même un LSTM n'extrait rien, c'est qu'il n'y a rien à extraire. C'est l'illusion de motif (§7.3) en version machine.*

Pièges & malentendus

- **Lire la baisse in-sample comme un succès**. C'est exactement le piège : seul l'OOS compte.
- **« hit@6 du LSTM > hasard, donc il marche »**. Bruit d'échantillon (§5.7) ; le log-loss et le test apparié tranchent dans l'autre sens.
- **Croire qu'entraîner plus longtemps aidera**. Cela ne fait qu'aggraver l'écart $\text{train} \leftrightarrow \text{test}$.

Pour vérifier ta compréhension

1. Comment se manifeste le mirage dans les courbes train/OOS du LSTM ?
2. Pourquoi, sous H_0 , toute baisse de la perte in-sample est-elle « de la variance pure » ?
3. Le LSTM passe sous le plancher in-sample. Pourquoi n'est-ce pas une découverte ?

(Réponses [à la fin.](#))

Pour aller plus loin

- **Compromis biais-variance** : le cadre classique de la généralisation.

- **Régularisation & early stopping** : arrêter avant que l'écart ne se creuse.
 - **Double descente** : raffinement moderne de la courbe capacité $\leftrightarrow$ erreur.
 - **Lien §7.3** : le mirage comme aboutissement de l'illusion de motif.
-

Réponses aux exercices

5.1

1. Parce qu'inclure t ferait entrer la cible (le tirage t) dans la feature $\rightarrow$ fuite : le modèle « verrait » partiellement la réponse.
2. La **normalisation globale** (moyenne/écart-type sur train + test, §5.2), la sélection de features ou le réglage d'hyperparamètres sur tout le jeu.
3. Parce que le plancher est indépassable sous H_0 : le battre OOS trahit une fuite, un bug ou du sur-apprentissage, pas une vraie découverte.

5.2

1. Parce que la moyenne globale incorpore les données du **test** (le futur) : c'est une fuite qui invalide l'évaluation OOS.
2. Les statistiques **du train seul** ($\hat{\mu}_{\text{train}}, \hat{\sigma}_{\text{train}}$), appliquées telles quelles au test.

5.3

1. Parce que cumfreq/rollfreq agrègent le passé ; mélanger placerait des tirages futurs dans le train, qui fuiraient dans ces features $\rightarrow$ leakage.
2. Train **188 846** lignes (≤ 2020), test **25 578** (> 2020) ; séparés par une **coupure temporelle**, sans shuffle.

5.4

1. $\frac{d}{dq} \mathbb{E}_p[\ell] = -(p/q - (1-p)/(1-q)) = 0 \Rightarrow p(1-q) = q(1-p) \Rightarrow q = p$; dérivée seconde $> 0 \rightarrow$ minimum.
2. Pour éviter $\ln 0 = -\infty$: une prédiction $q = 0$ (ou 1) erronée enverrait le log-loss à l'infini ; on borne $q \in [\varepsilon, 1 - \varepsilon]$.
3. Le log-loss moyen est l'**opposé de la log-vraisemblance de Bernoulli** (à un facteur N près) : minimiser l'un = maximiser l'autre.

5.5

1. $H(6/49) = -6/49 \ln 6/49 - 43/49 \ln 43/49 \approx 0,3718 \text{ nat} = 0,3718/\ln 2 \approx 0,536 \text{ bit}$.
2. Parce qu'elle prédit le **vrai taux de base** $p = 6/49$: son log-loss attendu est exactement $H(p)$, le minimum (KL = 0).
3. Un sur-apprentissage (mémorisation in-sample) ou une fuite/un bug : sous H_0 , c'est impossible de façon **stable** en OOS.

5.6

1. Parce que $(q - y)^2 \in [0,1]$ (avec $q, y \in [0,1]$), tandis que $\ln q \rightarrow -\infty$ quand $q \rightarrow 0$.
2. Que la conclusion (les modèles ne battent pas le plancher) est robuste au choix de la règle de score propre — deux juges indépendants concordent.

5.7

1. $6 \times \frac{6}{49} \approx 0,735$: chacun des 6 numéros du top a, sous H_0 , une proba 6/49 d'être tiré, et on en somme 6 (espérance par linéarité, §0.4).
2. Parce que hit@6 **oscille fortement** (0,67–0,77) sur ~522 tirages → bruit d'échantillon ; le log-loss et le test apparié ($p < 0,0001$), robustes, montrent le LSTM **pire**.

5.8

1. Car $\text{Var}(d) = \text{Var}(\ell^A) + \text{Var}(\ell^B) - 2\text{Cov}$, et la covariance est forte (mêmes tirages) → la difficulté commune s'annule, variance réduite.
2. **Non** : significatif mais dans le **mauvais sens** — le MLP (0,3721) est *pire* que la constante (0,3718). « Significatif » ≠ « meilleur ».
3. Parce que Wilcoxon est **non paramétrique** (sur les rangs) : pas besoin de supposer la normalité des différences de pertes.

5.9

1. Parce que sous H_0 la vraie probabilité est 6/49 pour tous ; prédire cette constante minimise le log-loss attendu (atteint le plancher). Aucun modèle ne peut faire mieux.
2. Que la logistique n'apporte **aucun** gain : avec $p = 0,14$, son écart à la constante est du bruit (et elle est même très légèrement pire).

5.10

1. $\sigma(0) = 1/2$. Cohérent avec une absence d'information poussant vers une proba neutre ; le biais β_0 recale vers le vrai taux de base.
2. Parce que le modèle est **libre** d'utiliser gap (l'ancienneté) et **choisit** un poids ≈ 0 : l'ancienneté n'a aucun pouvoir prédictif $\rightarrow$ pas de « numéro dû ».
3. Estimer la logistique par **maximum de vraisemblance** revient à **minimiser le log-loss** (entropie croisée) : ce sont les deux faces d'un même objectif.

5.11

1. Parce qu'il **pourrait** capter un signal non-linéaire s'il existait ; n'en trouvant aucun (collé au plancher), il **confirme** l'absence de signal — un résultat attendu et informatif.
2. Qu'un MLP à couche cachée assez large approxime toute fonction continue ; à double tranchant car cette capacité peut aussi **mémoriser le bruit** (§5.14).
3. Parce qu'il ne reçoit que **5 features faibles** sans information exploitable ; il n'y a rien à mémoriser (contrairement au LSTM nourri de la séquence complète).

5.12

1. Que la feature ne porte **aucune information prédictive** : la casser ne dégrade pas la performance OOS.
2. Parce qu'elle mesure l'**usage réel en prédiction OOS**, robuste aux corrélations, alors qu'un gros coefficient peut n'avoir aucune valeur prédictive (number_value).

5.13

1. Un vecteur $\in \{0,1\}^{49}$ avec un 1 pour chaque numéro tiré, soit **6 uns** (tirage sans remise, §0.3).
2. Parce qu'il a **beaucoup plus de capacité** (milliers de paramètres, mémoire récurrente) et reçoit la **séquence brute complète** — assez pour mémoriser des coïncidences, là où le MLP n'avait que 5 features faibles.

5.14

1. La log-loss **train plonge sous le plancher** (0,3659) tandis que la **OOS remonte au-dessus** (0,3754) et continue de grimper : l'écart train $\leftrightarrow$ test se creuse.
2. Parce que sous H_0 il n'y a **aucun signal à apprendre** ; toute baisse in-sample correspond à l'ajustement au bruit (variance), qui ne se transfère pas au test.
3. Parce que c'est de la **mémorisation**, pas de la généralisation : l'OOS, lui, est *pire* que le plancher (test apparié $p < 0,0001$). In-sample ne prouve rien. ``